

Algorytmy i Struktury Danych
Kolokwium 1 (27.03.2025)

Format rozwiązań

Wysłać należy tylko jeden plik: `kol1.py`

Plik można wysłać wielokrotnie, liczy się ostatnia wersja zapisana w systemie.

Rozwiązanie zadania musi się składać z krótkiego opisu algorytmu (wraz z uzasadnieniem poprawności) oraz jego implementacji. Zarówno opis algorytmu jak i implementacja powinny się znajdować w tym samym pliku Pythona (rozszerzenie `.py`). Opis powinien być na początku pliku w formie komentarza (w pierwszej linii w komentarzu powinno być imię i nazwisko studenta). Opis nie musi być długi—wystarczy kilka zdań, jasno opisujących ideę algorytmu. Implementacja musi być zgodna z szablonem kodu źródłowego dostarczonym wraz z zadaniem. Niedopuszczalne jest w szczególności:

1. korzystanie z wbudowanych funkcji sortujących,
2. korzystanie z zaawansowanych struktur danych (np. słowników czy zbiorów),
3. zmienianie nazwy funkcji implementującej algorytm, listy jej argumentów, lub nazwy pliku z rozwiązaniem,
4. wypisywanie na ekranie jakichkolwiek napisów innych niż wypisywane przez dostarczony kod (ew. napisy dodane na potrzeby diagnozowania błędów należy usunąć przed wysłaniem zadania).

Dopuszczalne jest natomiast:

1. korzystanie z następujących elementarnych struktur danych: krotka, lista.

Wszystkie inne algorytmy lub struktury danych wymagają implementacji przez studenta. Dopuszczalne jest oczywiście implementowanie dodatkowych funkcji pomocniczych w pliku z szablonem rozwiązania.

Zadania niezgodne z powyższymi ograniczeniami otrzymają ocenę 0 punktów. Rozwiązania w innych formatach (np. `.ZIP`, `.PDF`, `.DOC`, `.PNG`, `.JPG`) z definicji nie będą sprawdzane i otrzymają ocenę 0 punktów, nawet jeśli będą poprawne.

Testowanie rozwiązań

Żeby przetestować rozwiązanie zadania należy wykonać polecenie: `python kol1.py`

Szablon rozwiązania:	<code>kol1.py</code>
Złożoność oportunistyczna (0.5pkt):	$O(n \log n)$, gdzie n to liczba elementów w T .
Złożoność akceptowalna (2.0pkt):	$O(n + M/D)$, gdzie n to liczba elementów w T .
Złożoność wzorcowa (4.0pkt):	$O(n)$, gdzie n to liczba elementów w T .

Plugosław i Rzepisław – synowie sołtysa – znani są w swojej wsi z podejmowania wszystkich decyzji w sposób losowy. Aby nauczyć się fachu, otrzymali od ojca fragment pola przylegającego do drogi gminnej. Fragment drogi wzdłuż której znajduje się pole ma długość M . Wraz z polem dostali również kombajn, a każdy z braci otrzymał po N palików przeznaczonych do ogrodzenia pola od strony drogi.

Na początek, synowie podzielili fragment drogi na dwie części, tak aby mogli pracować niezależnie. Miejsce podziału p zostało oczywiście wybrane losowo z przedziału $(0, M)$. Następnie Plugosław zajął się stawianiem palików wzdłuż drogi na fragmencie $(0, p)$, a Rzepisław zajął się fragmentem (p, M) . Każdy z braci wybrał lokalizację dla swoich N palików na fragmencie drogi wzdłuż którego budował ogrodzenie losowo, zgodnie z rozkładem jednostajnym. Gdy wszystkie paliki zostały już wbite, bracia uświadomili sobie, że muszą zapewnić możliwość wjazdu kombajnem na pole. Ponieważ pojazd ten jest dość szeroki, może przejechać jedynie pomiędzy takimi dwoma sąsiadującymi palikami, których odległość wynosi co najmniej D . Bracia chcieliby mieć wiele różnych możliwości wjazdu, aby każdego dnia móc wybierać trasę losowo. Pomóż im znaleźć liczbę par sąsiadujących palików, między którymi kombajn zmieści się aby przejechać z drogi na pole. Musisz uwzględnić fakt, że z niejasnych powodów bracia przemieszczali dostarczoną listę położeń palików.

Proszę zaimplementować funkcję `ogrodzenie(M, D, T)`, która przyjmuje następujące argumenty:

- M – długość drogi, wzdłuż której bracia budowali ogrodzenie,
- D – minimalną szerokość odstępu między palikami, umożliwiającą przejazd kombajnu,
- T – tablicę o rozmiarze $2N$ zawierającą pomieszaną lokalizację palików (bracia losowali pozycje palików zgodnie z rozkładem jednostajnym, ale w tablicy T te pozycje zostały jeszcze dodatkowo pomieszaną; nie wiadomo zgodnie z jakimi kryteriami).

Każda wartość w tablicy T należy do zakresu $(0, M)$. Funkcja powinna zwrócić liczbę par sąsiadujących palików z T , między którymi odległość wynosi co najmniej D .

Przykład. Dla wejścia:

```
M = 10
D = 0.9
T = [3.55, 7.12, 1.3, 0.6]
```

wywołanie `ogrodzenie(M,D,T)` powinno zwrócić wartość 2 ponieważ kombajn może zmieścić się jedynie między palikami 3.55 i 7.12, oraz palikami 1.3 i 3.55.

Algorytm powinien być możliwie jak najszybszy. Proszę uzasadnić poprawność zaproponowanego algorytmu oraz oszacować jego złożoność czasową i pamięciową.

Algorytmy i Struktury Danych
Kolokwium 2 (08.05.2025)

Format rozwiązań

Wysłać należy tylko jeden plik: `kol2.py`

Plik można wysłać wielokrotnie, liczy się ostatnia wersja zapisana w systemie.

Rozwiązanie zadania musi się składać z krótkiego opisu algorytmu (wraz z uzasadnieniem poprawności) oraz jego implementacji. Zarówno opis algorytmu, jak i implementacja powinny się znajdować w tym samym pliku Pythona (rozszerzenie `.py`). Opis powinien być na początku pliku w formie komentarza (w pierwszej linii w komentarzu powinno być imię i nazwisko studenta). Opis nie musi być długi—wystarczy kilka zdań, jasno opisujących ideę algorytmu. Implementacja musi być zgodna z szablonem kodu źródłowego dostarczonym wraz z zadaniem. Niedopuszczalne jest w szczególności:

1. zmienianie nazwy funkcji implementującej algorytm, listy jej argumentów, lub nazwy pliku z rozwiązaniem,
2. wypisywanie na ekranie jakichkolwiek napisów innych niż wypisywane przez dostarczony kod (ew. napisy dodane na potrzeby diagnozowania błędów należy usunąć przed wysłaniem zadania).

Dopuszczalne jest natomiast:

1. korzystanie z następujących elementarnych struktur danych: krotka, lista, kolejka `collections.deque`, kolejka priorytetowa (`queue.PriorityQueue`),
2. korzystanie z wbudowanych funkcji sortujących (można założyć, że mają złożoność $O(n \log n)$).

Wszystkie inne algorytmy lub struktury danych (w tym słowniki) wymagają implementacji przez studenta. Dopuszczalne jest oczywiście implementowanie dodatkowych funkcji pomocniczych w pliku z szablonem rozwiązania.

Zadania niezgodne z powyższymi ograniczeniami otrzymają ocenę 0 punktów. Rozwiązania w innych formatach (np. `.ZIP`, `.PDF`, `.DOC`, `.PNG`, `.JPG`) z definicji nie będą sprawdzane i otrzymają ocenę 0 punktów, nawet jeśli będą poprawne.

Testowanie rozwiązań

Żeby przetestować rozwiązanie zadania należy wykonać polecenie: `python kol2.py`

Szablon rozwiązania:	<code>kol2.py</code>
Złożoność akceptowalna (2.0pkt):	$O(RE \log V)$
Złożoność wzorcowa (4.0pkt):	$O(E \log V)$
Gdzie V to liczba miast, E to liczba połączeń lotniczych, a R to liczba ośrodków.	

Stary Kontynent składa się z miast ponumerowanych od 0 do $V - 1$, pomiędzy którymi istnieje sieć połączeń lotniczych. W niektórych miastach znajdują się ośrodki wypoczynkowe. Grzegorz postanowił odwiedzić wszystkie te ośrodki – po jednym podczas każdej podróży.

Grzegorz rozpoczyna każdą podróż w swoim mieście rodzinnym. Korzystając z siatki połączeń między miastami, pośrednio lub bezpośrednio trafia do wybranego ośrodka, a następnie wraca lotami powrotnymi po tej samej trasie. Połączenia lotnicze między miastami są dwukierunkowe i mają przypisany z góry określony dodatni koszt – cena biletu jest identyczna w obu kierunkach. Koszt pokonania całej trasy jest równy sumie cen biletów za poszczególne loty na trasie. Aby nie popaść w rutynę, wyznaczając trasę do danego ośrodka, Grzegorz nigdy nie chce, aby przebiegała ona przez miasta, w których są odwiedzone już wcześniej ośrodki. Grzegorz jest jednak oszczędny i zawsze wybierze możliwie najtańszą pod względem kosztu trasę spełniającą te wymagania.

Na cel pierwszej podróży Grzegorz wybiera ośrodek, do którego koszt podróży jest najniższy. Po każdej kolejnej podróży jako następny cel wybiera nieodwiedzony jeszcze ośrodek, do którego prowadzi najtańsza możliwa trasa, która nie przebiega przez miasta z odwiedzionymi już ośrodkami. Jeśli istnieje kilka ośrodków o jednakowym, najniższym koszcie, Grzegorz wybiera ten o najniższym indeksie. Czynność tę powtarza dopóki nie odwiedzi wszystkich ośrodków, lub gdy dotarcie do nieodwiedzonych jeszcze ośrodków nie będzie możliwe (na przykład dlatego, że wszystkie możliwe trasy do nieodwiedzonych ośrodków będą prowadziły przez ośrodki już odwiedzone).

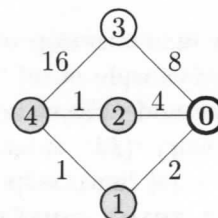
Proszę zaimplementować funkcję `lets_roll(start_city, flights, resorts)`, która zwróci sumaryczny koszt wszystkich podróży Grzegorza. Funkcja przyjmuje:

- `start_city` — Numer miasta rodzinnego Grzegorza.
- `flights` — Opis połączeń lotniczych jako lista krotek postaci (u, v, c) , gdzie u i v to indeksy miast między którymi istnieje połączenie lotnicze a c to dodatnia liczba oznaczająca cenę biletu dla tego połączenia.
- `resorts` — Listę numerów miast, w których są ośrodki wypoczynkowe. Miasto `start_city` na pewno nie znajduje się na liście ośrodków.

Algorytm powinien być możliwie jak najszybszy. Proszę uzasadnić poprawność zaproponowanego rozwiązania oraz oszacować jego złożoność czasową i pamięciową.

Przykład. Dla wejścia:

```
start_city = 0
flights = [(0, 1, 2), (0, 2, 4), (0, 3, 8),
           (3, 4, 16), (1, 4, 1), (2, 4, 1)]
resorts = [1, 2, 4]
```



Dla tych danych poprawną odpowiedzią jest liczba **60**.

Wyjaśnienie. Na pierwszą podróż Grzegorz wybierze ośrodek w mieście 1 i za bilety zapłaci $2 + 2$ (podróż do ośrodka nr 1 i z powrotem). Druga podróż odbędzie się do ośrodka w mieście 2 o koszcie 8. Trzecia podróż Grzegorza będzie do ośrodka w mieście 4, po jedynej dozwolonej trasie wiodącej przez miasto 3 o całkowitym koszcie 48. Sumaryczny koszt podróży wyniesie $4 + 8 + 48 = 60$.

Algorytmy i Struktury Danych

Kolokwium 3 (12.06.2025)

Format rozwiązań

Wysłać należy tylko jeden plik: `kol3.py`

Plik można wysyłać wielokrotnie, liczy się ostatnia wersja zapisana w systemie.

Rozwiązanie zadania musi się składać z krótkiego opisu algorytmu (wraz z uzasadnieniem poprawności) oraz jego implementacji. Zarówno opis algorytmu, jak i implementacja powinny się znajdować w tym samym pliku Pythona (rozszerzenie `.py`). Opis powinien być na początku pliku w formie komentarza (w pierwszej linii w komentarzu powinno być imię i nazwisko studenta). Opis nie musi być długi—wystarczy kilka zdań, jasno opisujących ideę algorytmu. Implementacja musi być zgodna z szablonem kodu źródłowego dostarczonym wraz z zadaniem. Niedopuszczalne jest w szczególności:

1. zmienianie nazwy funkcji implementującej algorytm, listy jej argumentów, lub nazwy pliku z rozwiązaniem,
2. wypisywanie na ekranie jakichkolwiek napisów innych niż wypisywane przez dostarczony kod (ew. napisy dodane na potrzeby diagnozowania błędów należy usunąć przed wysłaniem zadania).

Dopuszczalne jest natomiast:

1. korzystanie z następujących elementarnych struktur danych: krotka, lista, słownik, kolejka `collections.deque`, kolejka priorytetowa (`queue.PriorityQueue`),
2. korzystanie z wbudowanych funkcji sortujących (można założyć, że mają złożoność $O(n \log n)$).

Wszystkie inne algorytmy lub struktury danych (w tym słowniki) wymagają implementacji przez studenta. Dopuszczalne jest oczywiście implementowanie dodatkowych funkcji pomocniczych w pliku z szablonem rozwiązania.

Zadania niezgodne z powyższymi ograniczeniami otrzymają ocenę 0 punktów. Rozwiązania w innych formatach (np. `.ZIP`, `.PDF`, `.DOC`, `.PNG`, `.JPG`) z definicji nie będą sprawdzane i otrzymają ocenę 0 punktów, nawet jeśli będą poprawne.

Testowanie rozwiązań

Żeby przetestować rozwiązanie zadania należy wykonać polecenie: `python kol3.py`

Szablon rozwiązania:

Złożoność oportunistyczna (1.0pkt):

Złożoność akceptowalna (2.0pkt):

Złożoność wzorcowa (4.0pkt):

Gdzie n i m to wymiary blatu.

kol3.py

$O(n^2m)$, $O(nm^2)$

$O(nm)$

$O(n + m)$

Firma *Woodstack* produkuje deski parkietowe z drewnianego blatu o wymiarach $n \times m$ metrów. Maszyna produkcyjna odcina deski wyłącznie wzdłuż **górnej** lub **lewej** krawędzi blatu. Krótszy bok odciętej deski ma zawsze wymiar równy 1 metr. Każde cięcie musi ponadto przebiegać od krawędzi do krawędzi blatu – maszyna nie może przerwać cięcia w połowie. Po odcięciu deski, blat jest odpowiednio przesuwany a kolejne cięcia zawsze odbywają się względem pierwotnych krawędzi: **górnej** lub **lewej**.

Dla zachowania wysokiej jakości, żadna odcięta deska nie może zawierać więcej niż s sęków. Przed rozpoczęciem cięcia pracownicy zliczają więc sęki w każdym fragmencie blatu i zapisują wyniki w macierzy B o wymiarach $n \times m$. Suma komórek macierzy B przypadających na odciętą deskę nie może przekraczać s (zwróć uwagę, że każda deska będzie stanowił fragment kolumny lub wiersza w macierzy B – liczba sęków na desce to suma komórek w tym fragmencie). Komórka $B[0][0]$ macierzy odpowiada **górnemu, lewemu** narożnikowi blatu.

Wykonanie każdego cięcia jest kosztowne. Pracownicy chcą zatem zaplanować obróbkę blatu tak, aby wykonać minimalną możliwą liczbę cięć i aby żadna powstała deska nie miała więcej niż s sęków (pamiętaj, że fragment blatu pozostały po cięciach to zarazem ostatnia deska – on również musi mieć nie więcej, niż s sęków; jego krótszy bok musi mieć 1 metr). Celem wyznaczenia optymalnej kolejności cięć, pracownicy wpierw policzyli macierz sum cząstkowych C :

$$C[i][j] = \sum_{\substack{i \leq k < n \\ j \leq l < m}} B[k][l]$$

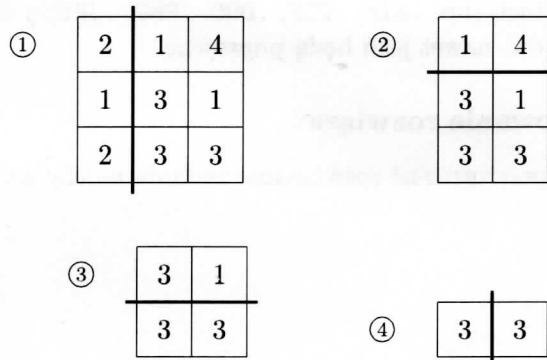
Element i, j macierzy C zawiera więc sumę komórki $B[i][j]$ oraz wszystkich komórek na prawo i w dół od niej.

Zaimplementuj funkcję `parkiet(B, C, s)` (gdzie B to tablica $n \times m$ zawierająca liczby sęków w komórkach blatu, C to tablica sum cząstkowych a s to limit na liczbę sęków w desce), która zwraca minimalną liczbę cięć niezbędną do pocięcia blatu zgodnie z warunkami zadania. Jeśli blat nie może być pocięty zgodnie z warunkami zadania, funkcja powinna zwrócić -1 .

Algorytm powinien być możliwie jak najszybszy. Proszę uzasadnić poprawność zaproponowanego rozwiązania oraz oszacować jego złożoność czasową i pamięciową.

Przykład. Dla wejścia:

$B = \begin{bmatrix} 2 & 1 & 4 \\ 1 & 3 & 1 \\ 2 & 3 & 3 \end{bmatrix}$
 $C = \begin{bmatrix} 20 & 15 & 8 \\ 13 & 10 & 4 \\ 8 & 6 & 3 \end{bmatrix}$
 $s = 5$



Poprawną odpowiedzią jest 4.

Wyjaśnienie.

Kolejne cięcia rozrysowane są na rysunku powyżej. W pierwszej kolejności ① wykonujemy cięcie pionowe odcinając deskę długości 3 wzdłuż lewej krawędzi o sumie sęków $2 + 1 + 2 = 5$. Kolejne dwa kroki to cięcia poziome ② i ③, które odcinają deski o sumach sęków odpowiednio $1 + 4 = 5$ i $3 + 1 = 4$. W ostatnim kroku ④ zostaje nam blat 2×1 którego przecinamy w pionie tworząc dwie ostatnie deski o wymiarach 1×1 , po 3 sęki każda. Liczba sęków na każdej z odciętych desek nie przekracza $s = 5$.

Szablon rozwiązania:	kolU.py
Złożoność podstawowa (1 pkt):	$O(n \cdot \log n)$, gdzie n to liczba elementów w tablicy.
Złożoność lepsza (3pkt):	$O(n \cdot k)$, gdzie n to liczba elementów w tablicy a k to maksymalna liczba kwiatów na kaktusie.
Złożoność wzorcowa (4pkt):	$O(n \cdot \log k)$, gdzie n to liczba elementów w tablicy a k to maksymalna liczba kwiatów na kaktusie.

W pewnej firmie n programistów postanowiło hodować kaktusy. Kaktus każdego programisty rośnie w osobnej doniczce. Doniczki są ponumerowane (jak to w zwyczaju u programistów) od zera do $n - 1$. Nadszedł czas kwitnienia roślin. Programiści umówili się, że dla każdej pary doniczek $i < j$, jeśli liczba kwiatów w doniczce i jest nie większa od liczby kwiatów w doniczce j to programista odpowiedzialny za doniczkę i wrzuca złotówkę do skarbonki „na cukierki”. W przeciwnym wypadku (liczba kwiatów w doniczce i jest większa od liczby kwiatów w doniczce j , przy czym $i < j$), programista odpowiedzialny za doniczkę i wrzuca złotówkę do skarbonki „na kawę”.

Wiadomo, że na każdym kaktusie jest od 1 do k kwiatów. Co więcej, maksymalna liczba kwiatów k jest stosunkowo niewielka ($k \ll n$). Jeden z programistów postanowił policzyć ile złotych wpadnie do skarbonki „na kawę”. Pomóż mu to zrobić.

Proszę zaimplementować funkcję **kawa**(T , k), która przyjmuje tablicę T zawierającą liczby kwiatów w doniczkach oraz maksymalną liczbę kwiatów k i zwraca wynikową liczbę złotych w skarbonce „na kawę”.

Algorytm powinien być możliwie jak najszybszy. Proszę uzasadnić poprawność zaproponowanego algorytmu oraz oszacować jego złożoność czasową i pamięciową.

Przykład. Dla wejścia:

$T = [2, 7, 8, 4]$

$k = 10$

wywołanie **kawa**(T , k) powinno zwrócić liczbę 2. Programista odpowiedzialny za doniczkę numer 3 musi wrzucić 2 złotówki do skarbonki „na kawę”.